

Advanced Programming

Project organization: data model and user interface

StringTokenizer, useDelimiter and split

Project organization

- The programs should be structured in a way that facilitates their development, expansion, and reuse of code and/or classes
- The separation of the code into its functional components facilitates:
 - Assigning parts to different team members
 - Independent evolution/updating of program parts without the need to change the whole project
 - Easier identification and correction of errors
 - ...

Project organization

- When solving the exercises, and whenever possible, the division should be as follows:
 - Data Model
 - data representation classes
 - methods for accessing and modifying data
 - associated logic and algorithms
 - User Interface: visualization and user interaction
 - mechanisms and classes that present model data in the most appropriate way
 - handling of commands or other types of user input that trigger model functions, resulting in data modification

Application code organization

- Rules to follow:
 - Do not place user interaction mechanisms (*print**, *Scanner*, ...) in the classes representing the model
 - Do not place data processing logic in the user interface classes

Exercise 10

- Write an application that implements the hangman game
- In the resolution of this exercise, the problem should be divided into independent classes for the data model and user interaction

Exercise 10 – classes

- The code should be structured in the following classes:
 - `HangmanGame`, which includes only the main function;
 - `HangmanDictionary`, which allows providing the words to use in the game. It should have only static members to store a table with the words, and two methods, one to obtain the quantity of stored words and another to get a word through its index;
 - `HangmanModel`, which will manage the entire game (without any user interaction code). It should contain methods that allow starting the game (picking randomly a new word), trying/verifying a letter, checking the end of the game, managing game progress information: number of attempts, already tried letters, ...;
 - `HangmanUI`, where all interaction with the user will be implemented.

Exercise 10 – main class

- "Example" of the HangmanGame class:

```
public class HangmanGame {  
    public static void main(String args[]) {  
        HangmanModel game = new HangmanModel();  
        HangmanUI gameUI = new HangmanUI(game);  
        gameUI.play();  
    }  
}
```

Exercise 10 – interaction example

Current situation:
Number of attempts: 0
Errors: 0 (max: 7)
Characters already attempted:

Attempt: U

Current situation:
Number of attempts: 1
Errors: 1 (max: 7)
Characters already attempted: U

Attempt: A

Current situation: ..A.
Number of attempts: 2
Errors: 1 (max: 7)
Characters already attempted: UA

Attempt: E

Current situation: .EA.
Number of attempts: 3
Errors: 1 (max: 7)
Characters already attempted: UAE

Attempt: O

Current situation: .EAO
Number of attempts: 4
Errors: 1 (max: 7)
Characters already attempted: UAEO

Attempt: P

Current situation: .EAO
Number of attempts: 5
Errors: 2 (max: 7)
Characters already attempted: UAEOP

Attempt: L
Congratulations! You guessed the word LEAO in 6 attempts

Exercise 10 – dictionary

- If another source cannot be found from which to obtain a set of words, the following set can be used:

"TOMORROW", "ARCHIMEDES", "LION", "TIGER", "ZEBRA", "PLATE",
"BROWN", "ORANGE", "PEA", "OTTER", "BALLAST", "ORANGUTAN",
"DISCIPLINE", "PROGRAMMING", "POTATO", "CLOWN", "BRAIN",
"TUNA", "PORTUGAL", "WILD", "ANIMAL", "IRRATIONAL",
"MATHEMATICS", "DISCREET", "EFFICIENT", "EFFECTIVE", "MARVEL",
"SYNOPSIS", "DEVICE", "LINEAR", "LEMON", "LAMP", "EAR",
"BUFFALO", "SHOE", "LOBSTER", "RISKY", "SCRATCHED",
"CARNIVAL", "BOLD", "RELEASED", "SIMPLE", "COMPLEX",
"STAPLER", "MONITOR", "KEYBOARD", "KEY", "CLOCK",
"HANDKERCHIEF", "WINDOW", "ROPE", "VIOLA", "GUITAR",
"POINTER", "ARGUER", "SAGACIOUS", "ERRONEOUS", "INSTITUTE",
"UPPER", "ENGINEERING", "DEPARTMENT", "INFORMATICS", "SYSTEM",
"CALL", "PRESIDENT", "FEBRUARY", "INCREASE", "SALARY",
"MONEY", "IMMEDIATE", "FLAMINGO", "RHINOCEROS",
"HIPPOPOTAMUS", "CODFISH", "PORCIE", "SARDINE", "SNAIL",
"INSECT", "FLYING", "ORANGE", "SPRINKLING", "EXTINCT",
"EXTERNAL", "AMBIGUOUS"

Exercise 10 – challenge

- Display the 'hangman' being drawn according to the progression represented below or another of your preference

0 Errors	1 Error	2 Errors	3 Errors	4 Errors	5 Errors	6 Errors	7 Errors

Exercise 11

- Define a class to represent information about a report
- A report (an instance of the `Report` class) is defined by the following fields:
 - *Title* (`String`)
 - *Set of authors* (`String []`)
 - *Text* (`StringBuilder` or `StringBuffer`)

Exercise 11

- The class should provide the following functionalities:
 - Add an author to the report ensuring that there are no duplicates
 - Remove an author, ensuring the maintenance of the order of the remaining ones
 - Add text. The text will be concatenated at the end of the existing text
 - Ensure that the first letters of the words after periods are in uppercase
 - Count the words in the text (words may be separated by more than one separator: spaces, tabs, line breaks, commas, and periods). The function should return the number of words
 - Count occurrences of a given word. The function should return the number of occurrences or 0 (zero) if the word does not exist in the text
 - Generate a `String` with all the report information (`toString()` method)
- Implement a *Java* application that allows testing all functionalities of the `Report` class.

Changing the size of an array

- After creating an array, its size cannot be changed
- If an array with more elements is needed, then a new array should be created with the desired size, and the elements from the original array should be copied

Changing the size of an array

```
int [] array = {1,2,3};  
System.out.println(Arrays.toString(array));  
    //output: [1, 2, 3]
```

```
int [] newArray = new int[array.length+2];  
System.arraycopy(array,0,newArray,0,array.length);  
array = newArray;  
System.out.println(Arrays.toString(array));  
    //output: [1, 2, 3, 0, 0]
```

```
array = Arrays.copyOf(array,array.length+2);  
System.out.println(Arrays.toString(array));  
    //output: [1, 2, 3, 0, 0, 0, 0]
```

Splitting a string into tokens

- To split the text of a `String` object, various techniques can be used:
 - Character-by-character analysis
 - `String.charAt`, `String.substring`, ...
 - Using a `Scanner` object to iterate over the string

```
Scanner sc = new Scanner(originalString);
sc.useDelimiter(<regex_with_desired_separators>);
while (sc.hasNext()) {
    String str = sc.next();
    ...
}
```

Splitting a string into tokens

- To split the text of a `String` object, various techniques can be used (*continuation*):

- Using the `split` method of the `String` class

```
String [] tokens =  
    originalString.split(<regex_separators>);
```

- Using a `StringTokenizer` object

```
StringTokenizer st = new StringTokenizer(  
    originalString,  
    <delimiter_string>  
);
```


Regular expressions

- Character sequences that allow describing patterns in texts
 - Information at:
 - <https://docs.oracle.com/javase/8/docs/api/java/util/regex/Pattern.html>
 - The `Pattern` class allows creating and compiling regular expressions
 - The `Matcher` class allows checking if a certain pattern is respected

```
Pattern p = Pattern.compile("ab");  
Matcher m = p.matcher("aaaaab");  
boolean b1 = m.matches(); // false  
boolean b2 = Pattern.matches("a+b", "aaaaab"); // true
```

- There are other classes and methods that indirectly use these classes, such as the `split` method and functionalities obtained by using the `useDelimiter` of the `Scanner`

Regular expressions

- Example to represent word separators (according to what is intended for the exercise):
`"[\\s, .]+"`
 - '`\\s`' represents default separators: *space, tab, line break, and page break*
 - The '+' indicates that one or more occurrences of these symbols/characters are acceptable (for example, there may be several spaces)
- Example of a regular expression to verify an IP address:

```
private static final String IPV4_PATTERN =  
    "^( [0-9] | [1-9] [0-9] | 1 [0-9] [0-9] | 2 [0-4] [0-9] | 25 [0-5] )\\.\\.\\. " +  
    " ( [0-9] | [1-9] [0-9] | 1 [0-9] [0-9] | 2 [0-4] [0-9] | 25 [0-5] )\\.\\. " +  
    " ( [0-9] | [1-9] [0-9] | 1 [0-9] [0-9] | 2 [0-4] [0-9] | 25 [0-5] )\\.\\. " +  
    " ( [0-9] | [1-9] [0-9] | 1 [0-9] [0-9] | 2 [0-4] [0-9] | 25 [0-5] ) $";
```